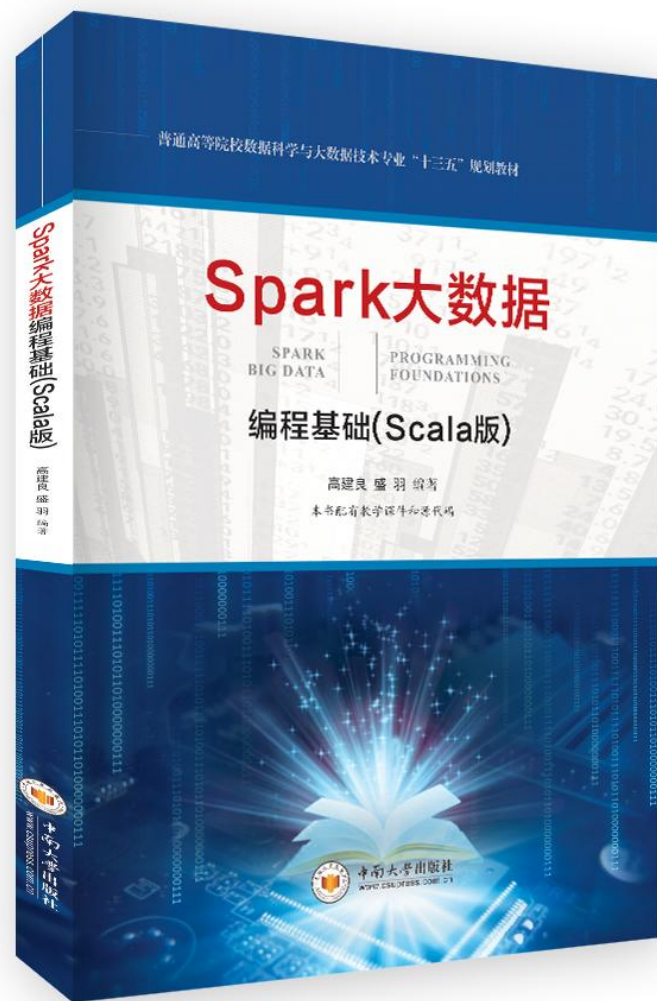
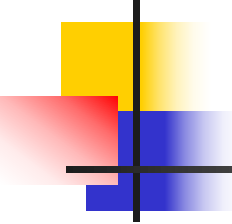




Spark大数据 编程基础 (Scala版)

<http://aibigdata.csu.edu.cn>

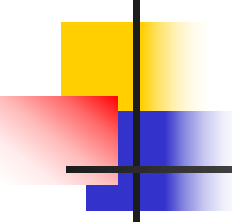




Spark 大数据编程基础（Scala版）

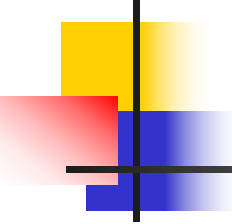
第七章 Spark Streaming

Spark Streaming属于流处理系统，它提供的处理引擎和RDD编程模型可以同时进行批处理与流处理。本章介绍了Spark Streaming实时数据流的数据处理。



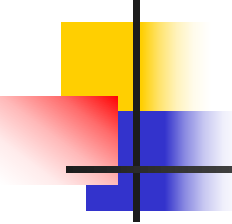
Spark 大数据编程基础（Scala版）

首先从Spark Streaming的工作机制入手，介绍了Spark Streaming的工作流程；接着介绍了Spark的核心数据抽象DStream以及操作，包括输入、转换、输出操作；然后结合不同的流式数据对Spark Streaming的操作进行进一步介绍；最后介绍Spark Streaming的性能优化方法。



主要内容

- 7.1 Spark Streaming工作机制
- 7.2 DStream输入源
- 7.3 DStream转换操作
- 7.4 DStream输出操作
- 7.5 Spark Streaming处理流式数据
- 7.6 Spark Streaming性能调优
- 7.7 本章小结



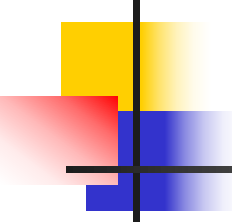
7.1 Spark Streaming工作机制

Spark Streaming属于核心Spark API的扩展，支持实时数据流的可扩展、高吞吐、容错的流处理。本节介绍Spark Streaming的工作机制，并引出Spark Streaming的核心概念DStream。

7.1.1 Spark Streaming 工作流程

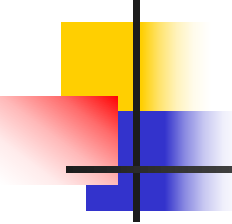


图7-1 Spark Streaming数据输入/输出图



7.1.1 Spark Streaming工作流程

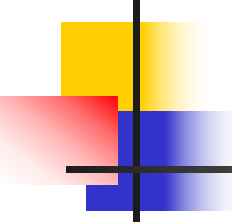
Spark Streaming对输入的流式数据进行处理，输出处理后的结果。如图7-1所示，Spark Streaming可以接受来自Kafka、Flume、HDFS/S3、Kinesis、Twitter或者TCP套接字的数据源等。然后，使用map、reduce、join、window等操作组成的算法进行处理，处理的结果可以输出到文件系统、数据库、现场Dashboards等。



7.1.1 Spark Streaming 工作流程

■ 批处理间隔

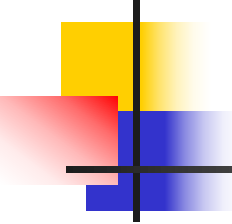
在Spark Streaming中，数据采集是逐条进行的，而数据的处理是按批进行的，因此在Spark Streaming中会先设置好批处理间隔（**batch interval**）。当超出批处理间隔时就会把采集到的数据汇集起来成为一批数据交给系统处理。



7.1.1 Spark Streaming工作流程

- **StreamingContext**对象

在RDD编程中需要生成一个SparkContext对象，在Spark SQL中需要生成一个SparkSession对象，同理，如果要运行一个Spark Streaming程序，就需要生成一个StreamingContext对象，它是Spark Streaming程序的主入口。



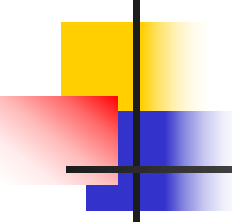
7.1.1 Spark Streaming工作流程

■ StreamingContext对象

可以从一个SparkConf对象中创建一个StreamingContext对象。在spark-shell中，由于默认了一个SparkContext对象，也就是sc，因此，可以用代码7-1创建StreamingContext对象：

代码7-1

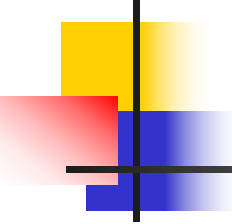
```
import org.apache.spark.streaming._  
val ssc = new StreamingContext(sc,Seconds(1))
```



7.1.1 Spark Streaming工作流程

- **StreamingContext**对象

StreamingContext(sc,Seconds(1))的两个参数中，sc表示SparkContext对象，Seconds(1)表示在对Spark Streaming的数据流进行分段时，每一秒切成一个分段，但是该系统无法实现毫秒级别的分段，因此，Spark Streaming无法实现毫秒级别的流计算。



7.1.1 Spark Streaming工作流程

- **StreamingContext**对象

如果是编写一个独立的Spark Streaming程序，则需要
在代码文件中用代码7-2创建StreamingContext对象。

代码7-2

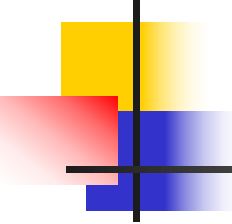
```
import org.apache.spark._  
import org.apache.spark.streaming._  
val conf = new SparkConf().setAppName("DStream").setMaster("local[2]")  
val ssc = new StreamingContext(conf,Seconds(1))
```

7.1.2 Spark Streaming处理机制

Spark Streaming在内部的处理机制是：先接收实时流的数据（input data stream），并按照一定的时间间隔切成分批的数据（batches of input data），然后把数据传输给Spark Engine进行处理，最终得到处理后的分批结果数据（batches of processed data）。如图7-2所示。

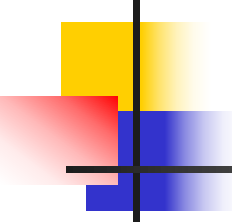


图7-2 Spark Streaming的工作原理



7.1.2 Spark Streaming处理机制

Spark Streaming是将流计算分解成一系列短小的批处理作业。批处理引擎是Spark Core，也就是把Spark Streaming的输入数据按照批处理间隔（如0.5秒）分解成一段一段数据（DStream），每一段数据都转换成Spark中的RDD，然后将Spark Streaming中对DStream的Transformation操作变成对RDD的Transformation操作，将RDD经过操作变成中间结果保存在内存中，整个流式计算根据业务需求可以对中间的结果进行叠加或者存储到外部设备。



7.1.2 Spark Streaming处理机制

■ DStream

DStream是Spark Streaming提供的基本抽象，它表示连续的数据流，可以是数据源接收的输入数据流，也可以是通过转换输入流生成的已处理数据流。在内部，DStream由一系列连续的RDD表示，这是Spark对不可变分布式数据集的抽象。

7.1.2 Spark Streaming处理机制

■ DStream

DStream中的每个RDD都包含来自特定时间间隔的数据，如图7-3所示。DStream是一个没有边界的集合，也就是它没有大小的限制，它代表的是一个时空的概念，图7-3中就能体现出多个时间段。



图 7-3 DStream与RDD的关系

7.1.2 Spark Streaming处理机制

■ DStream

Spark Streaming应用程序中，除了使用数据源产生的数据流来创建DStream，也会在已有的DStream上使用某种操作创建新的DStream，图7-4显示了对lines DStream做flatMap操作产生新的words DStream。

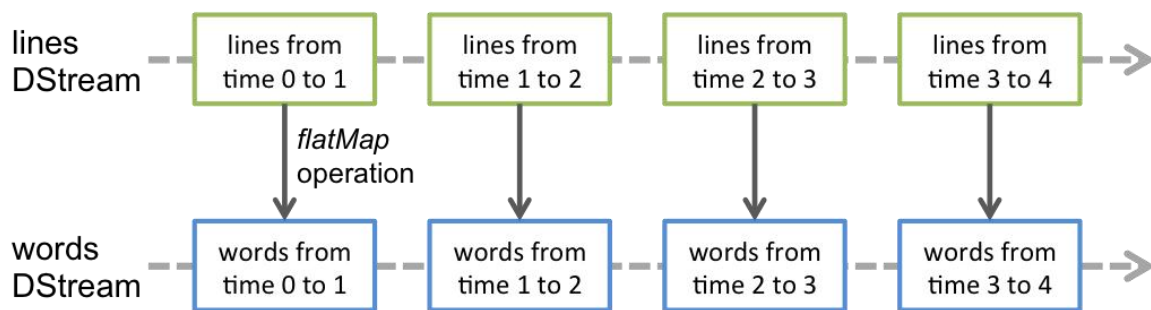
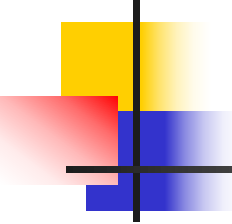


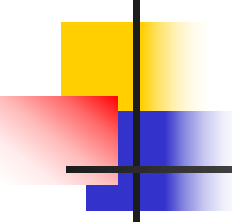
图 7-4利用flatMap操作把行DStream转换为单词DStream的过程



7.2 DStream输入源

在Spark Streaming中所有的操作都是基于流，而输入源是这一系列操作的起点，Spark Streaming提供基础和高级两种输入源，基础输入源和高级输入源，它们称为Input DStream。

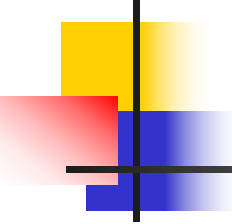
Input DStream是DStream的一种，它是流数据源中获取的原始数据流。



7.2 DStream输入源

■ Input DStream

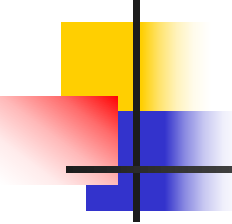
每个Input DStream（除了文件流之外）都会对应一个单一的接收器对象，该接收器对象从数据源接收数据并且存入Spark的内存中进行处理。每个Input DStream都会接收一个单一的数据流。在Streaming应用程序中，可以创建多个Input DStream并行接收数据流。



7.2 DStream输入源

- **Input DStream**

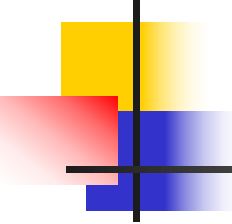
每个接收器是一个长期运行在Worker或者Executor上的任务，因此它将占用分配给Spark Streaming应用程序的一个核，因此，为了保证接收器能够接收数据，必须给Spark Streaming应用程序足够的核。



7.2 DStream输入源

- **Input DStream**

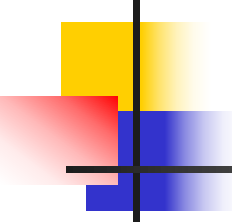
每个Input DStream（除了文件流之外）都会对应一个单一的接收器对象，该接收器对象从数据源接收数据并且存入Spark的内存中进行处理。每个Input DStream都会接收一个单一的数据流。



7.2 DStream输入源

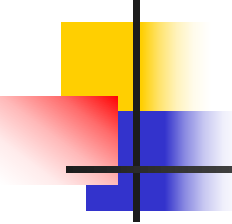
■ Input DStream

注意，运行本地模式时，当Master的URL设置为“Local”模式时，那么将会只有一个核来运行任务，在这种情况下，当程序只有一个Input DStream接收数据，接收器将独占这个核，因此程序将没有多余的核对数据进行其他操作。



7.2.1 基础输入源

基础输入源是指能够直接应用于StreamingContext API的输入源。例如文件流、套接字流、**RDD**队列流。

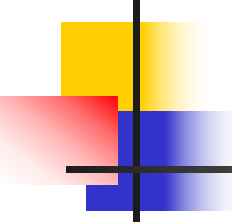


7.2.1 基础输入源

■ 文件流

文件流从兼容于HDFS API的文件系统（如HDFS、S3、NFS）中读取文件中的数据，它会对文件系统上的某个目录进行监听，一旦发现有新的文件生成，SparkStreaming就会自动把文件内容读取，文件流的DStream创建方法为：

`StreamingContext.fileStream[KeyClass, ValueClass, InputFormatClass]`

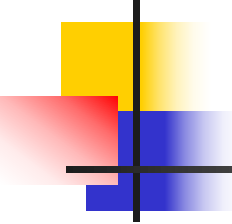


7.2.1 基础输入源

- 文件流

Spark Streaming将监控dataDirectory目录，并处理在该目录中创建的任何文件。对于文本文件而言，DStream的创建方法还可以是：

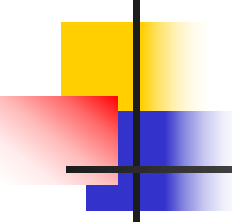
```
StreamingContext.textFileStream(dataDirectory)
```



7.2.1 基础输入源

■ 文件流

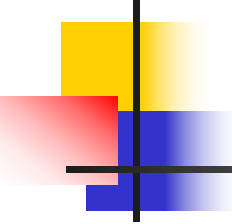
文件流并不需要运行接收器，因此不需要分配核。所有的 `DStream` 都是使用 `streamingContext.start()` 来处理数据，使用 `streamingContext.awaitTermination()` 等待处理被终止（手动或者由于任何错误），使用 `streamingContext.stop()` 来手动停止处理。



7.2.1 基础输入源

■ 套接字流

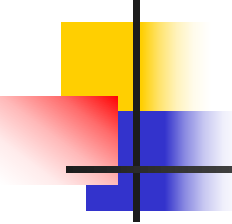
任何用户在用**Socket**（套接字）通信之前，首先要先申请一个**Socket**号，**Socket**号相当于该用户的电话号码。同时要知道对方的**Socket**，相当于对方也有一个电话号码。然后向对方拨号呼叫，相当于发出连接请求。对方假如在场并空闲，相当于通信的另一主机开机且可以接受连接请求，拿起电话话筒，双方就可以正式通话，相当于连接成功。



7.2.1 基础输入源

■ 套接字流

双方通话的过程，是**Socket**发送数据和从**Socket**接受数据的过程，相当于向电话机发出信号和从电话机接受信号。通话结束后，一方关闭**Socket**，相当于挂起电话机，撤销连接。所以套接字流是通过监听**Socket**端口接收的数据，相当于**Socket**之间的通信。



7.2.1 基础输入源

- 套接字流

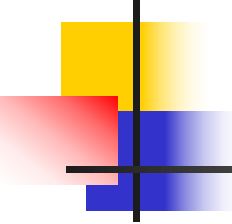
套接字流的创建方法如代码7-3所示：

代码7-3

```
//在本地创建一个 StreamingContext，分三个线程，批次间隔为 2s
val conf = new SparkConf().setMaster("local[3]").setAppName("WordCount")
val ssc = new StreamingContext(conf, Seconds(2))
//创建 DStream，连接 hostname:port，类似于 localhost:9999，之后对这个端口进行监听
val lines = ssc.socketTextStream("localhost", 9999)
```



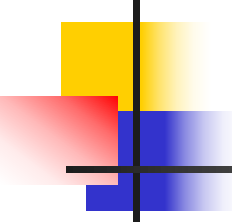
用nc -lk 9999打开该端口，然后可以向端口输入数据



7.2.1 基础输入源

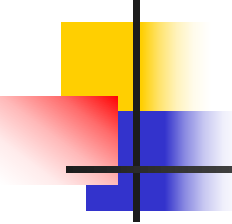
■ RDD队列流

在编写Spark Streaming应用程序时，可调用SparkContext对象的queueStream()方法创建基于RDD队列的Dstream。例如val inputStream = ssc.queueStream(rddQueue)中的rddQueue是RDD队列，inputStream是“RDD队列流”类型的数据源，可以通过对StreamingContext的设置调整批处理间隔，实现每隔一段时间从rddQueue队列中取出数据进行处理。



7.2.2 高级输入源

高级输入源依赖Spark不包含的库，如Kafka、Flume等，为了避免Spark和Kafka、Flume等库的依赖jar版本冲突的问题，Spark将这些输入源的创建方法实现在独立的库中。



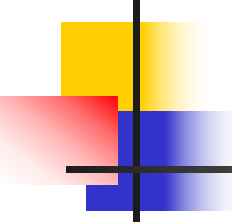
7.2.2 高级输入源

■ Kafka

(1) 添加spark-streaming-kafka-0-8_2.11的依赖，通过在工程的pom.xml（maven）中添加图 7-5所示内容。

```
<dependency>
  <groupId>org.apache.spark</groupId>
  <artifactId>spark-streaming-kafka-0-8_2.11</artifactId>
  <version>2.3.3</version>
</dependency>
```

图 7-5 添加Kafka依赖



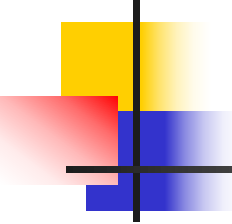
7.2.2 高级输入源

■ Kafka

- (2) 把Kafka目录中的libs目录添加到工程中。
- (3) 导入KafkaUtils类并创建基于Kafka的Dstream，如代码7-4所示，具体细节参考<https://spark.apache.org/docs/2.3.3/streaming-kafka-0-8-integration.html>

代码 7-4

```
import org.apache.spark.streaming.kafka._  
val kafkaStream = KafkaUtils.createStream(streamingContext, [ZK quorum], [consumer group  
id], [per-topic number of Kafka partitions to consume])
```



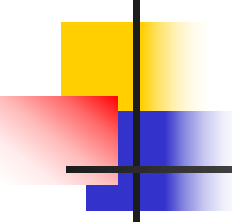
7.2.2 高级输入源

■ Flume

（1）添加spark-streaming-flume_2.11的依赖，通过在工程的pom.xml（maven）中添加如下内容。

```
<dependency>
  <groupId>org.apache.spark</groupId>
  <artifactId>spark-streaming-flume_2.11</artifactId>
  <version>2.3.3</version>
</dependency>
```

图 7-6 添加Flume依赖



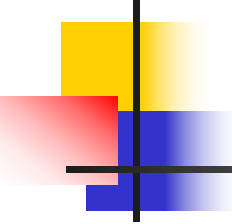
7.2.2 高级输入源

■ Flume

- (2) 把Flume目录中的lib文件夹添加到工程中。
- (3) 导入 FlumeUtils类并创建基于Flume的Dstream，如代码7-5所示，具体细节参考<https://spark.apache.org/docs/2.3.3/streaming-flume-integration.html>

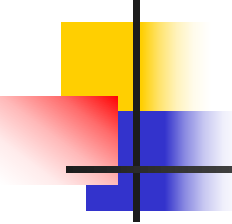
代码 7-5

```
import org.apache.spark.streaming.flume._  
val flumeStream = FlumeUtils.createStream(streamingContext, [chosen machine's hostname]  
[chosen port])
```



7.2.2 高级输入源

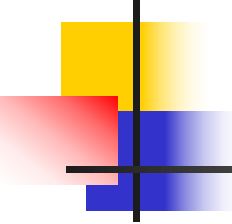
需要注意的是，这些高级的来源一般在**spark-shell**中不可用，因此基于这些高级来源的应用不能在**spark-shell**中进行测试，若需要在**spark-shell**中使用，需要下载相应的**Maven**工程**Jar**依赖，并添加到类路径中。表**7-1**是常见的输入源对应的项目表。



7.2.2 高级输入源

表 7-1 常见的输入源和对应的项目

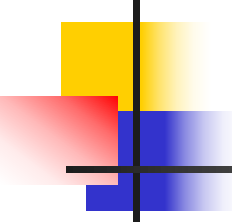
输入源名称	项目名称	描述
Kafka	Spark-streaming-kafka_2.11	可从 Kafka 中接收数据流
Flume	Spark-streaming-flume_2.11	可从 Flume 中接收数据流
Twitter	Spark-streaming-twitter_2.11	可从 TwitterUtils 工具类调用 Twitter4j，可得到公众的流
MQTT	Spark-streaming-mqtt_2.11	可从 MQTT 消息队列中接收流数据
ZeroMQ	Spark-streaming-zeromq_2.11	可从 ZeroMQ 消息队列中接收流数据



7.3 DStream转换操作

DStream支持两种操作，一种是转换操作，生成一个新的DStream，另一种是输出操作，可以把数据写入外部系统中。

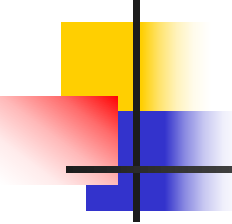
本节介绍DStream的转换操作。DStream的转换操作可以分为无状态和有状态两种。



7.3.1 无状态转换操作

- 无状态转换操作含义

在无状态转换操作中，每个批次的处理不依赖于之前批次的的数据，这样的操作与第五章讲到的**RDD**的转换操作类似，例如**map()**、**filter()**、**reduceByKey()**等，都属于无状态转换操作。无状态转换操作把简单的**RDD**转换操作应用到每个批次上，也就是转换**DStream**中的每个**RDD**。

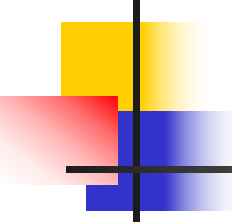


7.3.1 无状态转换操作

■ 无状态转换操作函数

表 7-2 无状态转换操作函数

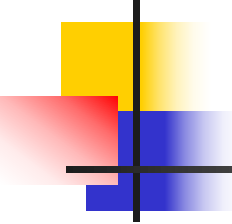
转换操作	含义
map(func)	将源 DStream 的每个元素传递给函数 func 返回一个新的 Dstream。
flatMap(func)	与 map 类似，但每个输入项可以映射到 0 个或多个输出项。
filter(func)	通过仅选择 func 返回 true 的源 DStream 的记录来返回新的 DStream。
repartition(numPartitions)	通过指定分区数更改 DStream 中的并行度级别。
union(otherStream)	返回一个新的 DStream，包含源 DStream 和其他 DStream 元素的集。
count()	对源 DStream 内部的所有 RDD 元素数量进行计数。



7.3.1 无状态转换操作

■ 无状态转换操作函数

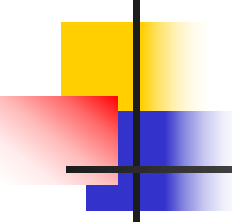
reduce(func)	使用函数 func（接受两个参数并返回一个结果）将源 DStream 中每个 RDD 的元素进行聚合，返回一个内部所包含的 RDD 只有一个元素的新的 DStream。
countByValue()	计算 DStream 中每个 RDD 内的元素出现的频次，并返回新的 DStream[(K,Long)]，其中 K 是 RDD 中元素的类型，Long 是元素出现的频次。
reduceByKey(func,[numTasks])	当调用一个(K,V)键值对类型的 DStream 时，返回一个新的(K,V)键值对类型的 DStream，这里键 K 不发生变化，而新的值 V 则是由 reduce 函数 func 聚合后得到的，这里的 numTasks 是可选参数，用来设置任务数量。



7.3.1 无状态转换操作

■ 无状态转换操作函数

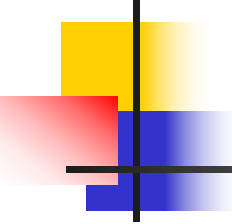
<code>join(otherStream,[numTasks])</code>	当在(K,V)和(K,W)键值对的两个 DStream 上调用时，返回(K, (V,W)) 键值对的新 DStream。
<code>cogroup(otherStream,[numTasks])</code>	当在(K,V)和(K,W)键值对的 DStream 上调用时，返回(K,Seq [V],Seq [W])元组的新 DStream。
<code>transform(func)</code>	通过对源 DStream 的每个 RDD 应用 RDD-to-RDD 函数返回一个新的 DStream，可以用来在 DStream 做任意 RDD 操作。



7.3.1 无状态转换操作

- 无状态转换操作

尽管这些函数看起来像作用在整个流上，但事实上每个 **DStream** 在内部是由许多 **RDD**（批次）组成，且无状态转换操作是分别应用到每个 **RDD** 上的。例如 `reduceByKey()` 会归约每个时间区间的数据，但是不会归约不同时间区间的数据。



7.3.2 有状态转换操作

- 有状态转换操作含义

DStream的有状态转换操作需要使用之前批次的数据或者是中间结果来计算当前批次的数据，是跨时间区间跟踪数据的操作，也就是说，一些先前批次的数据也被用来在新的批次中计算结果。主要的两种类型是基于窗口的操作和updateStateByKey操作，前者以一个时间阶段为滑动窗口进行操作，后者则用来跟踪每个键的状态变化。

7.3.2 有状态转换操作

- 基于窗口的操作

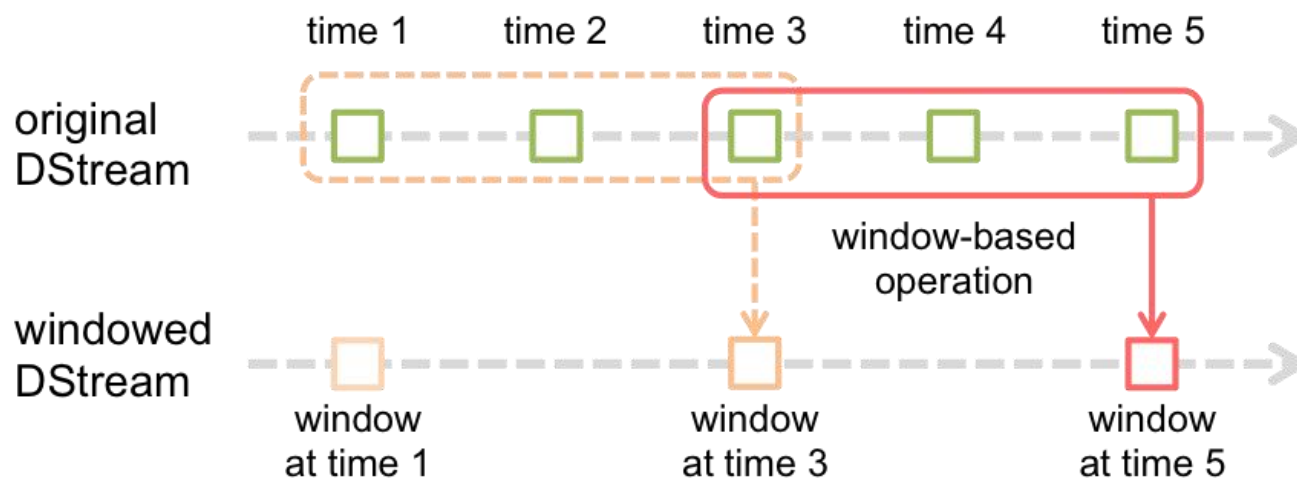
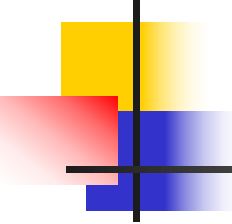


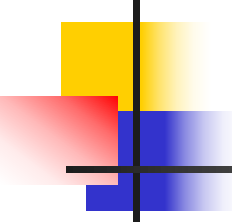
图 7-7 滑动窗口计算



7.3.2 有状态转换操作

■ 基于窗口的操作

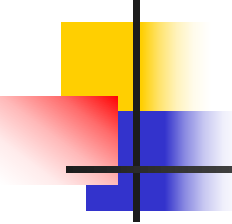
Spark Streaming提供了基于窗口的计算，允许通过滑动窗口对数据进行转换。图7-7表示窗口的滑动计算过程，当窗口在original DStream中按照定义的时间间隔滑动时，落入窗口的所有RDD（绿色正方形）被视为一个window，它们组成了windowed DStream。



7.3.2 有状态转换操作

■ 基于窗口的操作

任何窗口操作都需要制定两个参数，分别为窗口长度（窗口的持续时间），滑动步长（执行基于窗口操作计算的时间间隔），这两个参数必须是源DStream批处理时间间隔的倍数。

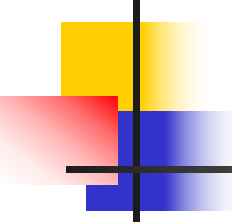


7.3.2 有状态转换操作

■ 基于窗口的操作函数

表 7-3 窗口转换操作函数

转换	含义
<code>window(windowLength,slideInterval)</code>	根据源 DStream 的窗口批次计算返回一个新的 DStream。
<code>countByWindow(windowLength,slideInterval)</code>	返回基于滑动窗口的 DStream 中元素的数量。
<code>reduceByWindow(func>windowLength,slideInterval)</code>	基于窗口对源 DStream 中元素进行聚合操作，返回新的 Dstream。
<code>reduceByKeyAndWindow(func>windowLength,slideInterval,[numTasks])</code>	基于滑动窗口对(K,V)键值对类型的 DStream 中的 V 按照 K 使用聚合函数 func 进行聚合操作，得到新的 Dstream。



7.3.2 有状态转换操作

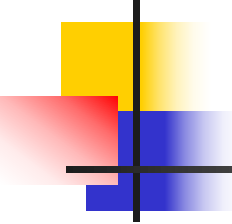
■ 基于窗口的操作函数

`reduceByKeyAndWindow(func,invFunc>windowLength,slideInterval,[numTasks])`

一个更高效的 `reduceByKeyAndWindow()` 的实现版本，先对滑动窗口中新的时间间隔内数据增量聚合并移去最早的与新增数据量时间间隔内的数据统计量。例如，计算 $m+4$ 时刻过去 5 秒窗口的 WordCount，可以将 $m+3$ 时刻过去 5 秒的统计量加上 $[m+4,m+5]$ 时刻的统计量，然后减去 $[m-2,m-1]$ 时刻的统计量，这种方法可以复用中间 3 秒的统计量，提高效率。

`countByValueAndWindow>windowLength,slideInterval,[numTasks])`

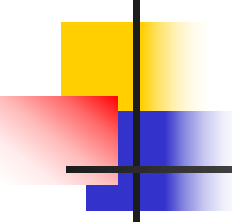
基于滑动窗口计算源 DStream 中每个 RDD 内每个元素出现的频次并返回 `DStream[(K,Long)]`，其中 K 是 RDD 中元素的类型，Long 是元素频次，与 `countByValue` 一样，reduce 任务的数量可以通过一个可选参数配置。



7.3.2 有状态转换操作

- 基于窗口的操作：**window()**操作

DStream最简单的窗口操作是**window()**，如图7-7所示返回新的DStream表示所请求的窗口操作的结果数据。**window()**生成的DStream中每个RDD会包含多个批次的数据，可对这些数据进行**count()**、**transform()**等操作。

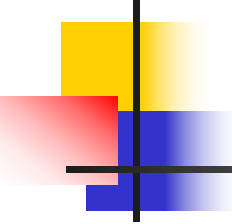


7.3.2 有状态转换操作

- 基于窗口的操作：window()操作

对滑动窗口的计算，假设批处理时间间隔为**10**秒，要创建一个最近**30**秒的时间间隔的窗口（最近**3**个批次），设置每两个批次计算一次窗口结果，应该把滑动步长设置为**20**秒，代码如下：

```
val inputStreamWindow = inputStream.window(Seconds(30),Seconds(20))
```



7.3.2 有状态转换操作

- 基于窗口的操作：**window()**操作

使用**window**操作对数据进行窗口化处理，设置监听间隔为**10s**，窗口大小为**30s**，滑动步长为**10s**。

输入数据如图7-8所示，数据中的每一行都是一个监听间隔内输入的数据。代码如代码7-6所示。

```
Spark Streaming  
Spark SQL  
Spark MLlib
```

图 7-8 输入数据

7.3.2 有状态转换操作

■ 基于窗口的操作：window()操作

```
import org.apache.spark.SparkConf
import org.apache.spark.streaming.{Seconds, StreamingContext}
object Windows_op {
  def main(arg: Array[String]): Unit = {
    val conf = new SparkConf().setMaster("local[2]").setAppName("Windowtest")
    //初始化 Streaming 对象并设置监听间隔为 10s
    val ssc = new StreamingContext(conf, Seconds(10))
    //用套接字流指定连接的 ip 端口
    //第一个参数为主机 IP，第二个为端口号
    val lines = ssc.socketTextStream("your IP", 9999)
    val words = lines.flatMap(_._split(" "))
    //指定窗口大小为 30s，滑动为 10s
    val windowwords = words.window(Seconds(30), Seconds(10))
    windowwords.print()
    ssc.start() //开始计算
    ssc.awaitTermination() //等待计算结束
  }
}
```

代码7-6

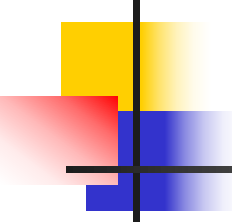
此处套接字流，后面的不会做说明

7.3.2 有状态转换操作

■ window()操作监听结果

Time: 1542336740000 ms		Time: 1542336760000 ms		Time: 1542336770000 ms
Spark	1	Spark		Spark
Streaming		Streaming		SQL
		Spark	3	Spark
Time: 1542336750000 ms		SQL		MLlib
Spark		Spark		Time: 1542336780000 ms
Streaming		MLlib		Spark
Spark	2			MLlib
SQL				5

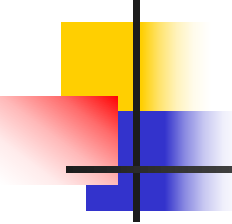
图 7-9 window操作输出结果



7.3.2 有状态转换操作

- **window()**操作监听结果

代码7-6将每组输入的数据按照空格分隔输出，由于窗口的大小为30s，滑动步长为10s，是窗口大小的三倍，所以最近30s内的输入数据会被处理输出，每次的数据会被监听三次，监听的结果如图7-9所示。



7.3.2 有状态转换操作

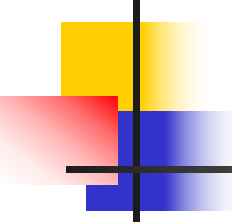
- 基于窗口的操作：**countByWindow()**操作

使用countByWindow操作统计窗口内的数据的数量，设置监听间隔为10s，窗口大小为30s，滑动步长为10s。

输入数据如图7-10所示，数据中的每一行都是一个监听间隔内输入的数据。代码如代码7-7所示。

```
1 2 3 4 5
6 7 8 9 10 11
12 13 14
```

图 7-10 输入数据

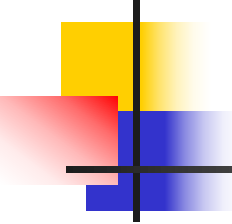


7.3.2 有状态转换操作

- 基于窗口的操作：countByWindow()操作

```
import org.apache.spark.SparkConf
import org.apache.spark.streaming.{Seconds, StreamingContext}
object countByWindow_op {
  def main(arg: Array[String]): Unit = {
    val conf = new SparkConf().setMaster("local[2]").setAppName("CountByWindowtest")
    val ssc = new StreamingContext(conf, Seconds(10))
    //设置检查点
    ssc.checkpoint("your checkpoint path ")
    val lines = ssc.socketTextStream("your IP", 9999)//第一个参数为主机 IP, 第二个为端口号
    val words = lines.flatMap(_.split(" "))
    //根据窗口大小统计 DStream 元素个数
    val windowwords = words.countByWindow(Seconds(30), Seconds(10))
    windowwords.print()
    ssc.start()
    ssc.awaitTermination()
  }
}
```

代码7-7

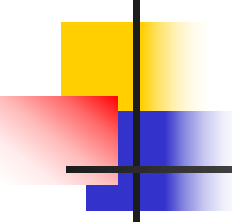


7.3.2 有状态转换操作

- **countByWindow()**操作监听结果

Time: 1542337880000 ms 5	1	Time: 1542337910000 ms 9	4
Time: 1542337890000 ms 11	2	Time: 1542337920000 ms 3	5
Time: 1542337900000 ms 14	3	Time: 1542337930000 ms 0	6

图 7-11 countByWindow操作输出结果



7.3.2 有状态转换操作

- 基于窗口的操作：reduceByKeyAndWindow()操作

第一种方法：

```
reduceByKeyAndWindow(func>windowLength,slideInterval,[numTasks])
```

第二种方法：

```
reduceByKeyAndWindow(func,invFunc>windowLength,slideInterval,[numTasks])
```

这两种方法都是对窗口内的数据的键值对形式进行reduceByKey操作，但是内部执行却有区别。假设有一个数据流执行reduceByKeyAndWindow操作，窗口大小为4s，滑动步长为1s，每秒钟有一个键值对输入，该操作的形式如图 7-12所示

7.3.2 有状态转换操作

- 基于窗口的操作：reduceByKeyAndWindow()操作

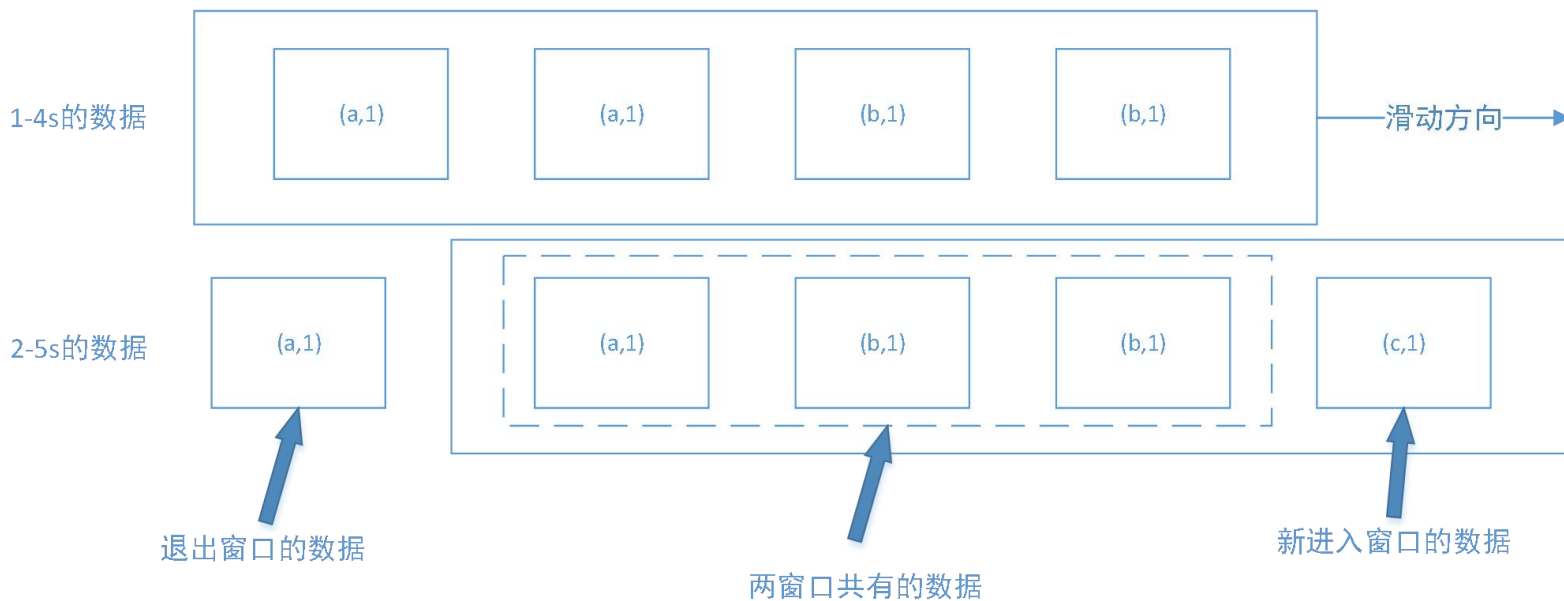
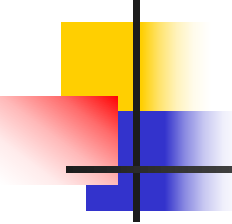


图 7-12 countByWindow操作输出结果

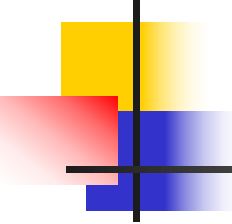


7.3.2 有状态转换操作

- 基于窗口的操作：**reduceByKeyAndWindow()**操作

第一种方法对第二个新窗口的数据进行**reduceByKey**的操作，当滑动到新的位置，需要对窗口内数据全部统计。

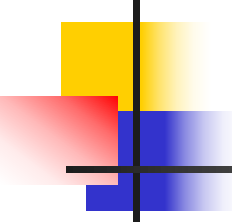
而第二种方法中参数内的两个函数是对新进入窗口的数据和退出窗口的数据进行操作的。



7.3.2 有状态转换操作

- 基于窗口的操作: `reduceByKeyAndWindow()`操作

如果进行单纯的词频统计, 采用第二种方法可以写成`reduceByKeyAndWindow(_+_, _-_, seconds(4), seconds(1), 2)`, 前面两个参数是对新进入的数据进行对应Key值的Value值相加, 对退出的数据对应的Key值的Value值相减, 而第一种方法只需要写成`reduceByKeyAndWindow((a:Int,b:Int) =>(a + b),Seconds(4),Seconds(1),2)`。



7.3.2 有状态转换操作

- 基于窗口的操作：`reduceByKeyAndWindow()`操作

然而用第二种方法的好处是利用了历史得到的词频统计的结果，只针对发生变化的部分进行计算，大大降低了计算开销。代码7-8是`reduceByKeyAndWindow`操作的代码。

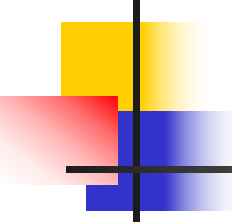
- 

```
import org.apache.spark.SparkConf
import org.apache.spark.streaming.{Seconds, StreamingContext}

object reduceByKeyAndWindow_op {
  def main(arg: Array[String]): Unit = {
    val conf = new SparkConf().setMaster("local[2]").setAppName("reduceByKeyAndWindowtest")

    val ssc = new StreamingContext(conf, Seconds(10))
    val lines = ssc.socketTextStream("your IP", 9999)//第一个参数为主机 IP，第二个为端口
    val words = lines.flatMap(_.split(" "))
    //第一种方法
    val WordCounts = words.map(x => (x, 1)).reduceByKeyAndWindow((a:Int,b:Int) => (a + b),Seconds(30),Seconds(10),2)
    //第二种方法 WordCounts = words.map(x => (x, 1)).reduceByKeyAndWindow(_+_._,Seconds(30),Seconds(10),2)
    WordCounts.print()
    ssc.start()
    ssc.awaitTermination()
  }
}
```

代码7-8



7.3.2 有状态转换操作

- 基于窗口的操作：**reduceByKeyAndWindow()**操作

代码设置窗口大小为**30s**，滑动步长为**10s**，启动两个**Task**线程，对窗口内的数据执行词频统计操作，输入数据如图 7-13所示，数据中的每一行都是一个监听间隔内输入的数据。



```
Spark Streaming  
Spark SQL  
Spark MLlib
```

图 7-13 输入数据

第一种方法的监听结果如图 7-14所示，第二种监听的结果如图7-15所示。

7.3.2 有状态转换操作

■ 第一种reduceByKeyAndWindow()监听结果

Time: 1542559010000 ms

(Streaming,1)
(Spark,1) **1**

Time: 1542559020000 ms

(SQL,1)
(Streaming,1) **2**
(Spark,2)

Time: 1542559030000 ms

(MLlib,1)
(SQL,1)
(Streaming,1) **3**
(Spark,3)

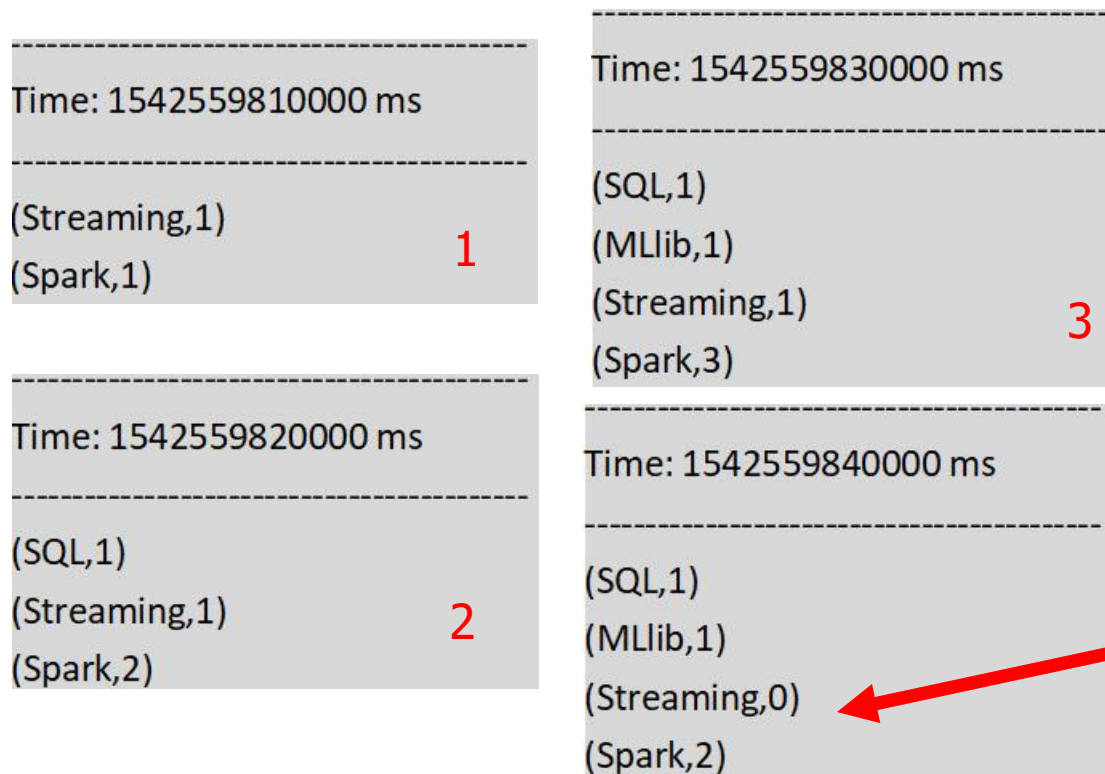
Time: 1542559040000 ms

(MLlib,1)
(SQL,1) **4**
(Spark,2)

图 7-14 第一种方法输出

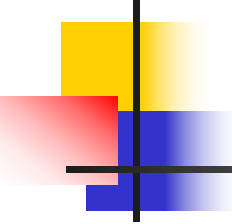
7.3.2 有状态转换操作

- 第二种reduceByKeyAndWindow()监听结果



曾经存在过的数据离开窗口，由于Key值存在，Value值变为0

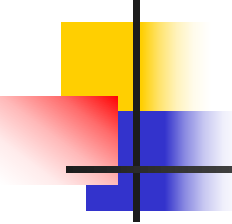
图 7-14 第二种方法输出



7.3.2 有状态转换操作

- **updateStateByKey**转换操作

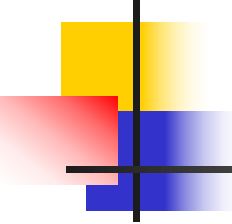
在跨批次之间维护状态时，需要使用**updateStateByKey**操作。该操作提供了状态变量的访问，用于键值对的形式**DStream**。给定一个由（键，事件）键值对构成的**DStream**，并传递根据新的事件更新每个键对应的状态函数，构建新的**DStream**，其内部数据为（键，状态）键值对。



7.3.2 有状态转换操作

- **updateStateByKey**转换操作

用单词词频统计的例子说明该情况，当处理每个批次的DStream时，如果是无状态的转换操作，单词处理时只对本批次内的单词进行词频统计，不会考虑之前的批次的单词，不同批次的单词词频是独立的，而对于**updateStateByKey**转换操作，本批次的词频统计会在之前批次的词频统计基础上进行不断累加，最终得到的词频是所有批次单词的词频统计结果。

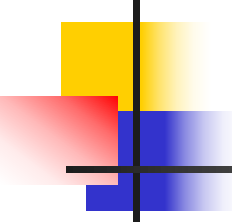


7.3.2 有状态转换操作

- **updateStateByKey**转换操作

当用updateStateByKey(updateFunc)方法输入时，参数updateFunc是一个函数，该函数定义为：

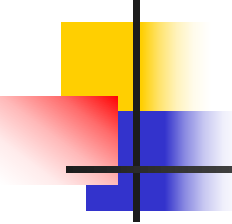
```
(Seq[V], Option[S]) => Option[S]
```



7.3.2 有状态转换操作

- **updateStateByKey**转换操作

其中V和S表示的是数据类型，如Int。updateFunc函数的第一个输入参数属于Seq[V]类型，表示当前Key对应的所有Value，第二个参数属于Option[S]类型，表示当前Key的历史状态，函数返回值类型Option[S]，表示当前Key的新状态。



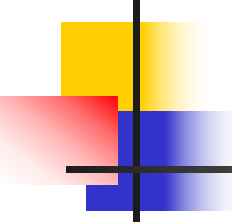
7.3.2 有状态转换操作

- **updateStateByKey**转换操作

updateStateByKey转换操作如代码7-9所示：

代码7-9

```
import org.apache.spark.SparkConf
import org.apache.spark.streaming.{Seconds, StreamingContext}
object updateStateByKey_op {
  def main(args: Array[String]) {
    val conf = new SparkConf().setMaster("local[2]")
      .setAppName("UpdateStateByKeyDemo")
    val ssc = new StreamingContext(conf, Seconds(20))
    //要使用 updateStateByKey 方法，必须设置 Checkpoint
    ssc.checkpoint("your checkpoint path")
    //第一个参数为主机 IP，第二个为端口号
    val socketLines = ssc.socketTextStream("your IP", 9999)
```

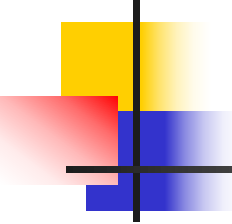



7.3.2 有状态转换操作

- **updateStateByKey**转换操作

代码7-9 (续)

```
socketLines.flatMap(_._split(" ")).map(word=>{word,1})
  .updateStateByKey(
    (currValues:Seq[Int],preValue:Option[Int]) =>{
      val currValue = currValues.sum      //对当前 Key 值对应的 Value 进行求和
      //对当前 Key 值对应的当前、历史状态 Value 值进行求和
      Some(currValue + preValue.getOrElse(0))
    }).print()
ssc.start()
ssc.awaitTermination()
ssc.stop()
}
```



7.3.2 有状态转换操作

- **updateStateByKey**转换操作

输入数据如图7-16所示，数据中的每一行都是一个监听间隔内输入的数据。

```
1 1 1 2 2
```

```
2 2 3 3 3
```

```
3 3 3 1 1
```

```
//第四次没有输入任何值
```

图7-16输入数据

7.3.2 有状态转换操作

■ **updateStateByKey**转换操作监听结果

Time: 1542352620000 ms

(2,2)

(1,3)

1

Time: 1542352640000 ms

(2,4)

(3,3)

(1,3)

2

Time: 1542352660000 ms

(2,4)

(3,6)

(1,5)

3

Time: 1542352680000 ms

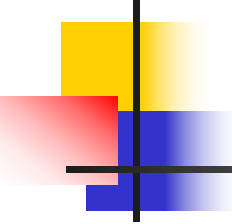
(2,4)

(3,6)

(1,5)

4

图7-17 updateStateByKey操作输出结果



7.4 DStream输出操作

输出操作是将DStream的数据输出到外部的系统。

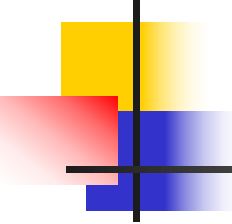
输出操作作用于DStream后，触发所有的DStream变换实际执行，这一点与RDD执行（Action）类似。表 7-4 列出了目前主要的输出操作。

7.4 DStream输出操作

■ 输出操作函数

表7-4 输出操作

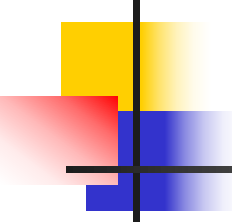
转换	描述
<code>print()</code>	在运行流应用程序的驱动程序节点上打印 DStream 中每批数据的前十个元素。
<code>savaAsTextFiles(prefix,[suffix])</code>	将此 DStream 的内容保存为文本文件，每个批处理时间间隔内产生的文件以"prefix-TIME_IN_MS[.suffix]"的方式命名。
<code>savaAsObjectFiles(prefix,[suffix])</code>	将此 DStream 的内容保存为序列化 Java 对象的 SequenceFiles，每个批处理时间间隔内产生的文件以"prefix-TIME_IN_MS[.suffix]"的方式命名。
<code>savaAsHadoopFiles(prefix,[suffix])</code>	将此 DStream 中的内容以文本的形式保存为 Hadoop 文件，每个批次处理时间间隔内产生的文件以 prefix-TIME_IN_MS[.suffix]的方式命名。
<code>foreachRDD(func)</code>	最基本的输出操作，将 func 函数应用于 DStream 中的 RDD，这个操作会输出数据到外部系统，比如保存 RDD 到文件或数据库等，该 func 函数是在运行 streaming 应用的 Driver 进程里执行的。



7.4 DStream输出操作

- **saveAsTextFiles**操作

使用**saveAsTextFiles**操作把Spark Streaming监听并处理后的结果以文本文件的形式保存到所指定的路径中，具体实现如代码 7-10所示。



7.4 DStream输出操作

- **saveAsTextFiles**操作

代码7-10

```
import org.apache.spark.SparkConf
import org.apache.spark.streaming.{Seconds, StreamingContext}
object saveAsTextFiles_op {
  def main(arg: Array[String]): Unit = {
    val conf = new SparkConf().setMaster("local[2]").setAppName("Windowtest")
    val ssc = new StreamingContext(conf, Seconds(10))
    val lines = ssc.socketTextStream("your IP", 9999)//第一个参数为主机 IP, 第二个为端口号
    //保存到要保存的路径下, 会自动生成 test+"-监听时间".txt 文件
    lines.saveAsTextFiles("your save path", "txt")
    ssc.start()
    ssc.awaitTermination()
  }
}
```

7.4 DStream输出操作

- **saveAsTextFiles**操作

代码7-10每隔十秒钟生成一个文件夹到指定路径下，文件夹名称为**test+“监听时间”+.txt**的文件，结果如图7-18所示：

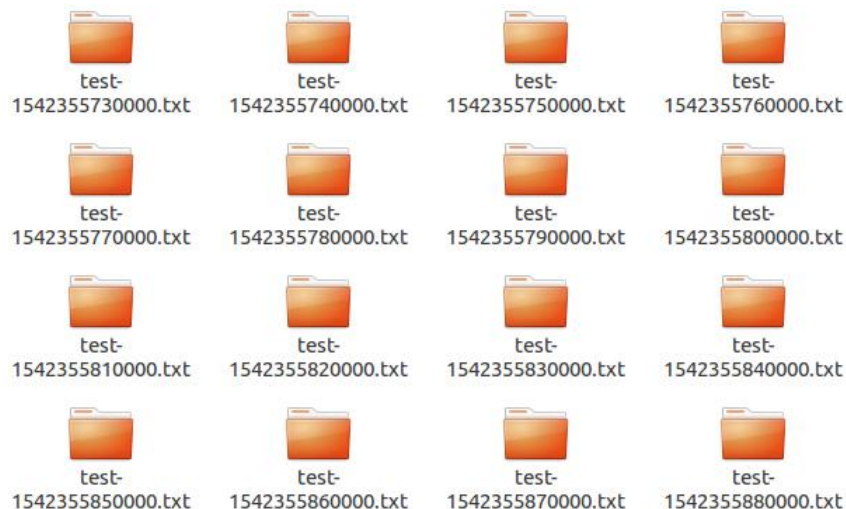


图7-18 监听结果文件夹

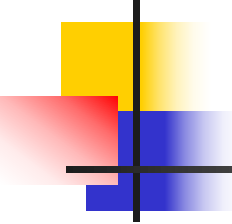
7.4 DStream输出操作

- **saveAsTextFiles**操作

当在监听端口输入Spark Streaming时，会在对应监听时间的文件夹内找到图7-19所示的文件，打开文件part-00000文件可以找到输入的内容。

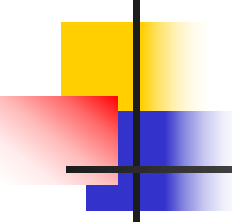


图7-19 文件夹内监听到的文件



7.5 Spark Streaming处理流式数据

本节介绍在**IDEA**中创建工程运行**Spark**示例，包括文件流、套接字流、**RDD**队列流以及**Kafka**消息队列流的**WordCount**示例。

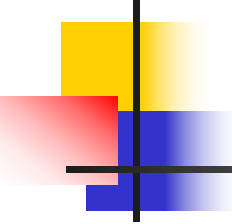


7.5.1 文件流

■ 文件流创建过程

(1) 新建一个文件夹用来接收数据文件，在该目录下的Spark_Streaming文件夹内新建一个file文件夹，作为监听的目录：

```
$ cd Spark_Streaming  
$ mkdir file
```



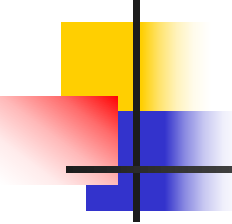
7.5.1 文件流

- 文件流创建过程

(2) 在工程中创建Streaming_one.scala文件，键入代码如代码7-11所示。

代码7-11

```
import org.apache.spark.SparkConf
import org.apache.spark.streaming.{Seconds, StreamingContext}
object Streaming_one {
  def main(arg: Array[String]): Unit = {
    val conf = new SparkConf().setMaster("local[2]").setAppName("testone")
    val ssc = new StreamingContext(conf, Seconds(30))//设置监听时间间隔为 30s
```

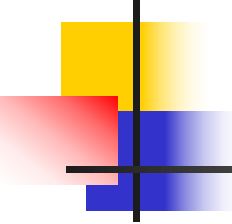


7.5.1 文件流

- 文件流创建过程

```
//监听的文件执行 wordcount
val words = lines.flatMap(_.split(" "))
val wordCounts = words.map(x=>(x,1)).reduceByKey(_+_ )
wordCounts.print()
ssc.start()
ssc.awaitTermination()
}
}
```

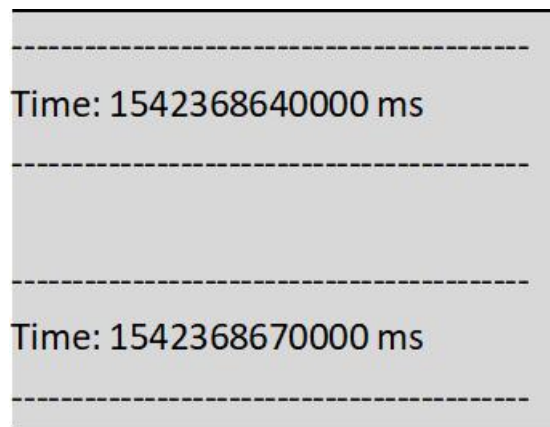
代码7-11 (续)



7.5.1 文件流

■ 文件流监听结果

①没有输入文件时的监听状态如图7-20所示：



```
-----  
Time: 1542368640000 ms  
-----  
  
-----  
Time: 1542368670000 ms  
-----
```

图7-20 无文件输入监听结果

7.5.1 文件流

■ 文件流监听结果

②有数据输入时的监听状态，即在路径下的文件夹中新建文本文件，并写入数据如图7-21所示：



```
Spark Streaming
Spark SQL
Spark MLlib
```

图7-21 输入数据

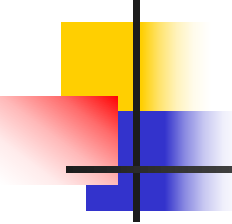
执行监听的结果如图7-22所示：



```
-----
Time: 1542368700000 ms
-----
```

```
(SQL,1)
(MLlib,1)
(Streaming,1)
(Spark,3)
```

图7-22 有文件输入时的监听结果



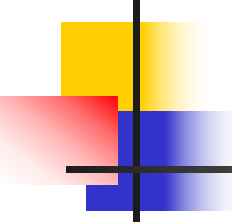
7.5.2 RDD队列流

■ RDD队列流创建过程

在工程中创建Streaming_two.scala文件，键入代码如代码7-12所示。

代码7-12

```
import org.apache.spark.SparkConf
import org.apache.spark.rdd.RDD
import org.apache.spark.streaming.{Seconds, StreamingContext}
object Streaming_two {
  def main(args:Array[String]){
    val sparkConf = new SparkConf().setAppName("RDDQueue").setMaster("local[2]")
    val ssc = new StreamingContext(sparkConf,Seconds(4))
    //创建 RDD 队列
    val rddQueue = new scala.collection.mutable.SynchronizedQueue[RDD[Int]]()
    val queue = ssc.queueStream(rddQueue) //创建输入的队列数据流
    //处理队列中的 RDD 数据为(数取余 10,1)的形式
    val map = queueStream.map(r=>(r%10,1))
```

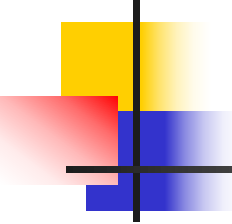



7.5.2 RDD队列流

■ RDD队列流创建过程

代码7-12 （续）

```
val reduce = map.reduceByKey(_+_)  
reduce.print()  
ssc.start()  
//创建和向队列推入 RDD  
for(i<-1 to 10){  
    //创建一个包含 1-100 元素的 RDD  
    //设置线程暂停一秒，目的是为了使每次循环有一个停止时间，让不至于循环结束太快  
    //只进行了一次监听。该操作十次循环会用十秒钟  
        rddQueue+=ssc.sparkContext.makeRDD(1 to 100)  
        Thread.sleep(1000)  
    }  
    ssc.stop() //结束 Streaming 程序  
}
```

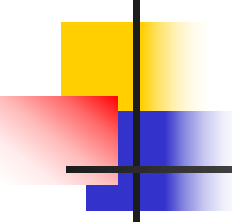


7.5.2 RDD队列流

- **RDD队列流结果与含义**

代码创建RDD队列rddQueue，队列包含1-100的整数。

接着使用queueStream(rddQueue)创建基于RDD队列的DStream，并对DStream中的数与10取余后进行WordCount操作，代码执行结果如图 7-23所示。

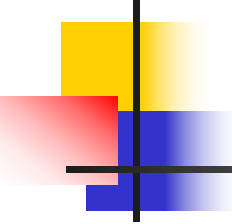


7.5.2 RDD队列流

■ RDD队列流结果与含义

Time: 1542370880000 ms	Time: 1542370884000 ms	Time: 1542370888000 ms
(4,10)	(4,10)	(4,10)
(0,10)	(0,10)	(0,10)
(6,10)	(6,10)	(6,10)
(8,10)	(8,10)	(8,10)
(2,10)	(2,10)	(2,10)
(1,10)	(1,10)	(1,10)
(3,10)	(3,10)	(3,10)
(7,10)	(7,10)	(7,10)
(9,10)	(9,10)	(9,10)
(5,10)	(5,10)	(5,10)
1	2	3

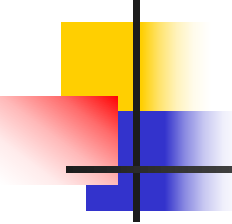
图 7-23 RDD队列流监听结果



7.5.2 RDD队列流

■ RDD队列流结果与含义

输出结果中对RDD队列监听了3次，是因为代码Thread.sleep(1000)会在该处执行休眠一秒种，十次循环休眠十秒后结束程序，而程序每隔4秒监听一次，所以会在第0、4、8秒处监听到消息，会有三次监听结果。



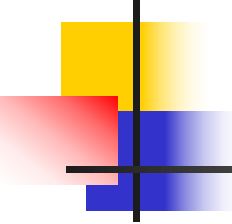
7.5.3 套接字流

■ 套接字流创建过程

(1) 在工程中创建Streaming_three.scala文件，键入代码如代码7-13所示

代码7-13

```
import org.apache.spark.SparkConf
import org.apache.spark.streaming.{Seconds, StreamingContext}
object Streaming_three {
  def main(arg: Array[String]): Unit = {
    val conf = new SparkConf().setMaster("local[2]").setAppName("NetworkWordCount")
    val ssc = new StreamingContext(conf, Seconds(10))
    val lines = ssc.socketTextStream("your IP", 9999)//第一个参数为主机 IP，第二个为端口号
    val words = lines.flatMap(_.split(" "))
    val wordcounts = words.map(x => (x, 1)).reduceByKey(_ + _)
    wordcounts.print()
    ssc.start()
    ssc.awaitTermination()
  }
}
```



7.5.3 套接字流

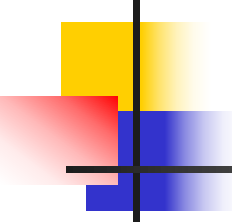
■ 套接字流创建过程

(2) 在终端窗口中输入`nc -lk 9999`，进入9999端口，运行代码，接着在端口输入图7-24所示数据，一个时间间隔内输入一条数据。



```
$ nc -lk 9999
Spark Streaming
Spark SQL
Spark MLlib
```

图7-24 输入数据



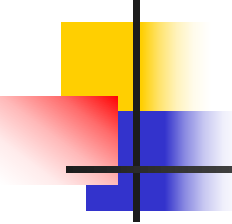
7.5.3 套接字流

■ 套接字流结果

对监听的数据执行WordCount操作，结果如图7-25所示：

Time: 1542373170000 ms	Time: 1542373180000 ms	Time: 1542373190000 ms
(Streaming,1)	(SQL,1)	(MLlib,1)
(Spark,1)	(Spark,1)	(Spark,1)

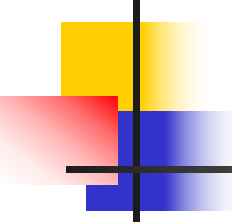
图7-25 套接字流监听结果



7.5.4 Kafka消息队列流

■ Kafka概述

Apache Kafka是分布式发布-订阅消息系统。发布/订阅 (Publish/Subscribe) 模式：使消息的分发可以突破目的队列指向的限制，使消息按照特定的主题甚至内容进行分发，用户或应用程序可以根据主题或内容接收到所需要的消息。



7.5.4 Kafka消息队列流

■ Kafka概述

发布/订阅功能使得发送者和接收者之间的耦合关系变得更为松散，发送者不必关心接收者的目的地址，而接收者也不必关心消息的发送地址，而只是根据消息的主题进行消息的收发。

7.5.4 Kafka消息队列流

■ Kafka概述

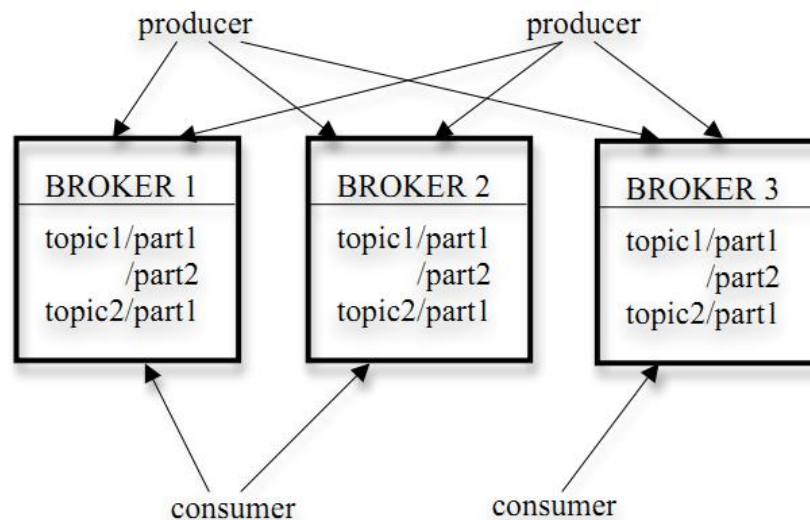
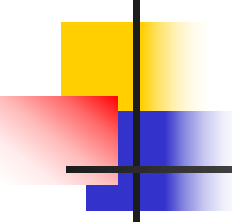


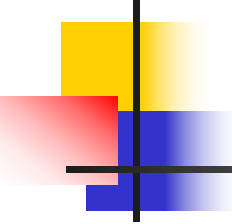
图7-26 Kafka的生产者\消费者示意图



7.5.4 Kafka消息队列流

■ Kafka概述

图7-26是Kafka的工作原理图，图中将消息的发布（Publish）称作Producer（生产者），将消息的订阅（Subscribe）表述为 Consumer（消费者），将中间的Kafka服务器称作 Broker(代理)。Producer生产数据，给定Topic，交给 Broker 进行存储，Consumer消费数据，根据Topic从Broker中取出数据，完成一系列对数据的处理操作。



7.5.4 Kafka消息队列流

- 安装与配置Kafka

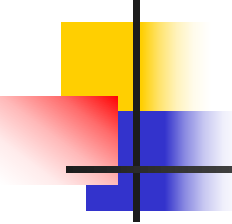
(1) 安装Kafka，一共分为两个步骤：

1)从官网上下载Kafka的安装包，在Linux终端输入命令：

```
$ wget http://mirrors.shu.edu.cn/apache/kafka/1.1.1/kafka_2.11-1.1.1.tgz
```

2)下载Kafka安装包完毕后，对其进行解压，输入命令：

```
$ tar zxvf kafka_2.11-1.1.1.tgz
```



7.5.4 Kafka消息队列流

- 安装于配置Kafka

(2) 对Kafka的安装进行测试:

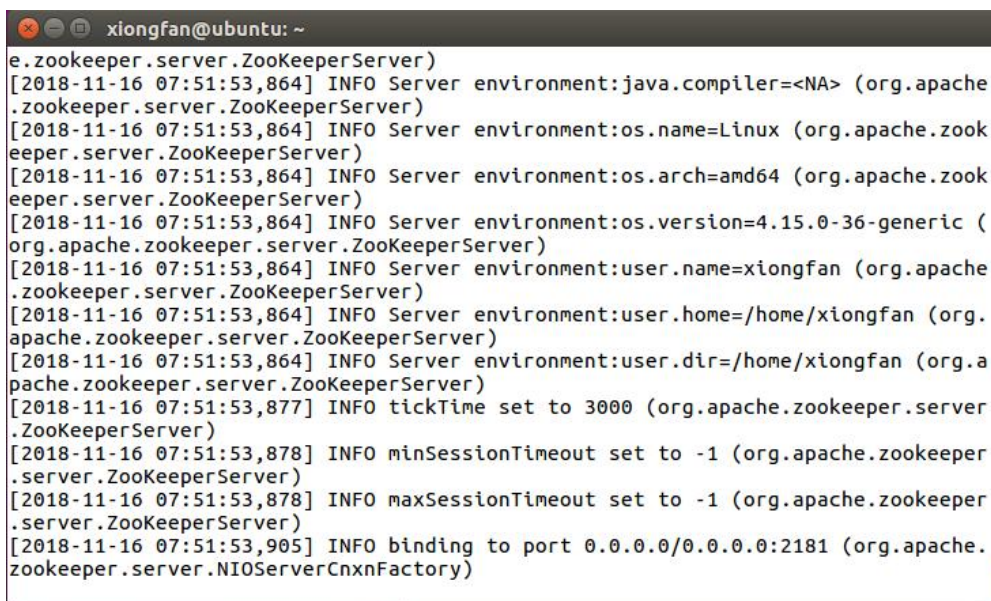
1)在解压后的Kafka文件夹下打开终端，执行命令来启动Zookeeper服务:

```
$ bin/zookeeper-server-start.sh config/zookeeper.properties
```

7.5.4 Kafka消息队列流

- 安装于配置Kafka

成功启动Zookeeper服务后的显示如图7-27所示：

A terminal window titled 'xiongfam@ubuntu: ~' displays the output of starting the Zookeeper service. The logs show various environment variables being set and configuration parameters being initialized. The final line indicates that the service is binding to port 0.0.0.0:2181.

```
xiongfam@ubuntu: ~
e.zookeeper.server.ZooKeeperServer)
[2018-11-16 07:51:53,864] INFO Server environment:java.compiler=<NA> (org.apache
.zookeeper.server.ZooKeeperServer)
[2018-11-16 07:51:53,864] INFO Server environment:os.name=Linux (org.apache.zook
eeper.server.ZooKeeperServer)
[2018-11-16 07:51:53,864] INFO Server environment:os.arch=amd64 (org.apache.zook
eeper.server.ZooKeeperServer)
[2018-11-16 07:51:53,864] INFO Server environment:os.version=4.15.0-36-generic (
org.apache.zookeeper.server.ZooKeeperServer)
[2018-11-16 07:51:53,864] INFO Server environment:user.name=xiongfam (org.apache
.zookeeper.server.ZooKeeperServer)
[2018-11-16 07:51:53,864] INFO Server environment:user.home=/home/xiongfam (org.
apache.zookeeper.server.ZooKeeperServer)
[2018-11-16 07:51:53,864] INFO Server environment:user.dir=/home/xiongfam (org.a
pache.zookeeper.server.ZooKeeperServer)
[2018-11-16 07:51:53,877] INFO tickTime set to 3000 (org.apache.zookeeper.server
.ZooKeeperServer)
[2018-11-16 07:51:53,878] INFO minSessionTimeout set to -1 (org.apache.zookeeper
.server.ZooKeeperServer)
[2018-11-16 07:51:53,878] INFO maxSessionTimeout set to -1 (org.apache.zookeeper
.server.ZooKeeperServer)
[2018-11-16 07:51:53,905] INFO binding to port 0.0.0.0/0.0.0.0:2181 (org.apache.
zookeeper.server.NIOServerCnxnFactory)
```

图7-27 执行Zookeeper服务成功

7.5.4 Kafka消息队列流

■ 安装于配置Kafka

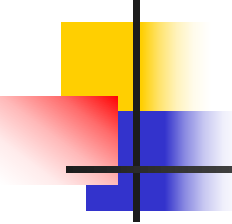
2)开启Kafka服务，在该目录下另起终端，执行代码如下：

```
$ bin/kafka-server-start.sh config/server.properties
```

成功启动Kafka服务后的显示如图7-28所示：

```
xiongf@ubuntu: ~/kafka_2.11-1.1.1
[2018-11-21 21:53:49,141] INFO [GroupCoordinator 0]: Startup complete. (kafka.co
ordinator.group.GroupCoordinator)
[2018-11-21 21:53:49,179] INFO [GroupMetadataManager brokerId=0] Removed 0 expir
ed offsets in 12 milliseconds. (kafka.coordinator.group.GroupMetadataManager)
[2018-11-21 21:53:49,201] INFO [ProducerId Manager 0]: Acquired new producerId b
lock (brokerId=0,blockStartProducerId=0,blockEndProducerId=999) by writing to Zk
with path version 1 (kafka.coordinator.transaction.ProducerIdManager)
[2018-11-21 21:53:49,287] INFO [TransactionCoordinator id=0] Starting up. (kafka
.coordinator.transaction.TransactionCoordinator)
[2018-11-21 21:53:49,293] INFO [TransactionCoordinator id=0] Startup complete. (
kafka.coordinator.transaction.TransactionCoordinator)
[2018-11-21 21:53:49,326] INFO [Transaction Marker Channel Manager 0]: Starting
(kafka.coordinator.transaction.TransactionMarkerChannelManager)
[2018-11-21 21:53:49,415] INFO [/config/changes-event-process-thread]: Starting
(kafka.common.ZkNodeChangeNotificationListener$ChangeEventProcessThread)
[2018-11-21 21:53:49,427] INFO [SocketServer brokerId=0] Started processors for
1 acceptors (kafka.network.SocketServer)
[2018-11-21 21:53:49,456] INFO Kafka version : 1.1.1 (org.apache.kafka.common.ut
ils.AppInfoParser)
[2018-11-21 21:53:49,456] INFO Kafka commitId : 8e07427ffb493498 (org.apache.kaf
ka.common.utils.AppInfoParser)
[2018-11-21 21:53:49,459] INFO [KafkaServer id=0] started (kafka.server.KafkaSer
ver)
```

图7-28 执行Kafka服务成功



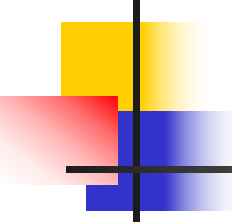
7.5.4 Kafka消息队列流

- 安装与配置Kafka

- (3) 配置Kafka依赖

成功启动这两个服务表明单机上的Kafka安装成功。

在Spark Streaming中应用Kafka的服务需要在编程环境中导入Kafka的依赖包。



7.5.4 Kafka消息队列流

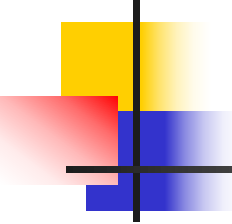
■ 安装与配置Kafka

(3) 配置Kafka依赖

1) 下载spark-streaming-kafka-0-8_2.11.jar包。在命令行输入：

```
$ wget http://central.maven.org/maven2/org/apache/spark/spark-streaming-kafka-0-8_2.11/2.3.3/spark-streaming-kafka-0-8_2.11-2.3.3.jar
```

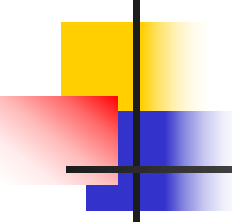
(注：Kafka的依赖包需要和Spark与Scala的版本相对应，具体版本对应的信息可查看<https://mvnrepository.com/artifact/org.apache.spark/spark-streaming-kafka-0-8>)



7.5.4 Kafka消息队列流

- 安装于配置Kafka

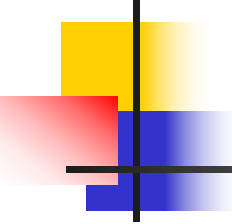
2)把spark-streaming-kafka-0-8_2.11.jar和Kafka目录下的libs文件夹中的jar包导入工程。



7.5.4 Kafka消息队列流

■ Kafka示例（处理过程概述）

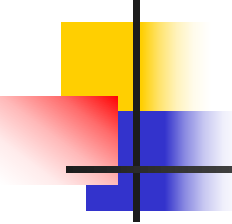
Producer生产了数据，会先通过Zookeeper找到Broker，然后将数据存放到Broker；Consumer根据其订阅的Topic消费数据，先通过Zookeeper找对应的 Broker，然后消费。模拟生产者每秒向Kafka发送4条消息，每条消息包含10个0~9的随机数，Spark Streaming作为消费者，使用reduceByKeyAndWindow()方法进行统计并显示结果。



7.5.4 Kafka消息队列流

- **Kafka**示例（创建过程）

（1）在工程中创建KafkaWordCountProducer.scala文件，以其作为生产者，向Kafka发送数据。键入代码如代码 7-14所示。

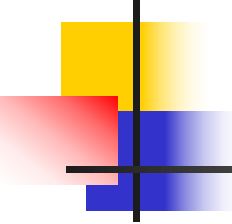


7.5.4 Kafka消息队列流

- **Kafka示例（创建过程）**

代码 7-14

```
import java.util.HashMap
import org.apache.kafka.clients.producer.{KafkaProducer, ProducerConfig, ProducerRecord}
object KafkaWordCountProducer {
  def main(args: Array[String]) {
    if (args.length < 4) {
      System.err.println("Usage: KafkaWordCountProducer <metadataBrokerList> <topic> " +
        "<messages> <words>")
      System.exit(1)
    }
    val Array(brokers, topic, messages, words) = args
    // Zookeeper 连接属性
    val props = new HashMap[String, Object]()
    props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, brokers)
    props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,
      "org.apache.kafka.common.serialization.StringSerializer")
  }
}
```

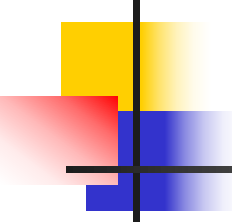


7.5.4 Kafka消息队列流

■ Kafka示例（创建过程）

代码 7-14 （续）

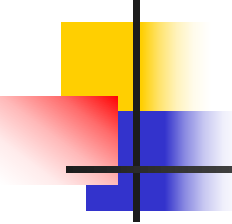
```
props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG,
    "org.apache.kafka.common.serialization.StringSerializer")
val producer = new KafkaProducer[String, String](props)
//发送信息，每秒一次，一次发送消息条数为 messages 条，每条消息有 words 个，每
//个是大于等于 0 小于 10 的随机数
while(true) {
    (1 to messages.toInt).foreach { messageNum =>
        val str = (1 to words.toInt).map(x => scala.util.Random.nextInt(10).toString)
            .mkString(" ")
        print(str)
        println()
        val message = new ProducerRecord[String, String](topic, null, str)
        producer.send(message)
    }
    Thread.sleep(1000)
}
}
```



7.5.4 Kafka消息队列流

- **Kafka示例（创建过程）**

（2）在工程中创建StreamingExamples.scala文件，用于设置log4j日志级别，该文件路径为Spark安装目录下的/examples/src/main/scala/org/apache/spark/examples/streaming，如代码 7-15所示：



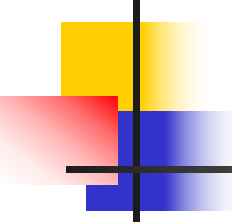
7.5.4 Kafka消息队列流

■ Kafka示例（创建过程）

代码 7-15

```
import org.apache.log4j.{Level, Logger}
import org.apache.spark.internal.Logging

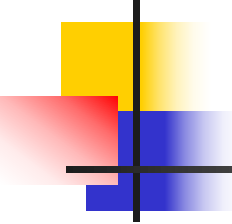
object StreamingExamples extends Logging {
  //如果用户尚未配置 log4j，为流设置合理的日志记录级别
  def setStreamingLogLevels() {
    val log4jInitialized = Logger.getRootLogger.getAllAppenders.hasMoreElements
    if (!log4jInitialized) {
      logInfo("Setting log level to [WARN] for streaming example." +
        " To override add a custom log4j.properties to the classpath.")
      Logger.getRootLogger.setLevel(Level.WARN)
    }
  }
}
```

7.5.4 Kafka消息队列流

- **Kafka**示例（创建过程）

（3）在工程中创建KafkaWordCountConsumer.scala文件，作为消费者从Kafka中获取数据进行处理，设置监听间隔为10秒，窗口大小为1分钟，滑动步长为10秒，执行reduceByKeyAndWindow操作，如代码7-16所示。



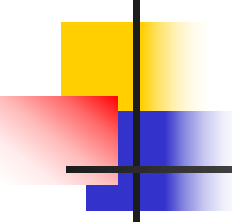
7.5.4 Kafka消息队列流

- **Kafka**示例（创建过程）

代码 7-16

```
import org.apache.spark.SparkConf
import org.apache.spark.streaming._
import org.apache.spark.streaming.kafka.KafkaUtils

object KafkaWordCountConsumer {
  def main(args:Array[String]){
    StreamingExamples.setStreamingLogLevels()
    val sc = new SparkConf().setAppName("KafkaWordCountConsumer").setMaster("local[2]")
    val ssc = new StreamingContext(sc,Seconds(10))
```

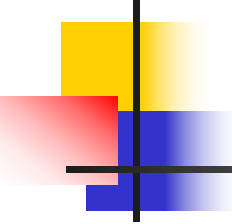


7.5.4 Kafka消息队列流

■ Kafka示例（创建过程）

代码 7-16 （续）

```
//设置检查点
ssc.checkpoint("your checkpoint path")
val zkQuorum = "localhost:2181" //Zookeeper 服务器地址
val group = " kafka_test" //topic 所在的 group
val topics = "sender" //topics 的名称
val numThreads = 1 //每个 topic 的分区数
val topicMap = topics.split(",").map(_._2).toMap
//Kafka 的 Dstream 配置
val lineMap = KafkaUtils.createStream(ssc,zkQuorum,group,topicMap)
val lines = lineMap.map(_._2)
val words = lines.flatMap(_._2.split(" "))
val pair = words.map(x => (x,1))
//对窗口内的 RDD 执行 reduceByKeyAndWindow 操作
val wordCounts = pair.reduceByKeyAndWindow((a:Int,b:Int) => (a + b),Minutes(1),Seconds(10),2)
wordCounts.print
ssc.start
ssc.awaitTermination
}
```



7.5.4 Kafka消息队列流

- **Kafka示例（创建过程）**

（4）设置生产者的配置参数，点击Run中的Debug Configurations，并在KafkaWordCountProducer中的program arguments输入信息如图7-29所示：

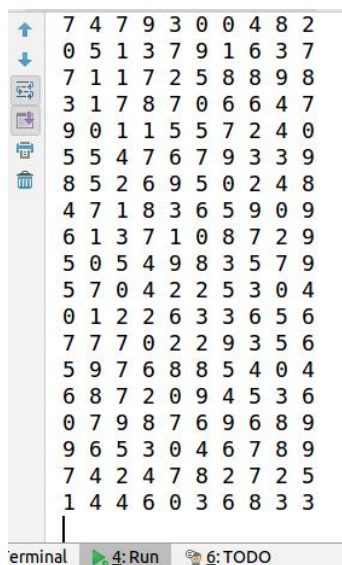
```
//对应 KafkaWordCountProducer 文件中的代码 val Array(brokers, topic, messages, words) = args
//中的参数，第一个参数是 Kafka 的服务器地址，第二个参数是 Topic 名称
localhost:9092 sender 4 10
```

图 7-29 配置参数图

7.5.4 Kafka消息队列流

■ Kafka示例（生产者生产数据）

（5）启动Zookeeper服务和Kafka服务后，运行KafkaWordCountProducer代码，得到随机数发送如图7-30所示。



```
7 4 7 9 3 0 0 4 8 2
0 5 1 3 7 9 1 6 3 7
7 1 1 7 2 5 8 8 9 8
3 1 7 8 7 0 6 6 4 7
9 0 1 1 5 5 7 2 4 0
5 5 4 7 6 7 9 3 3 9
8 5 2 6 9 5 0 2 4 8
4 7 1 8 3 6 5 9 0 9
6 1 3 7 1 0 8 7 2 9
5 0 5 4 9 8 3 5 7 9
5 7 0 4 2 2 5 3 0 4
0 1 2 2 6 3 3 6 5 6
7 7 7 0 2 2 9 3 5 6
5 9 7 6 8 8 5 4 0 4
6 8 7 2 0 9 4 5 3 6
0 7 9 8 7 6 9 6 8 9
9 6 5 3 0 4 6 7 8 9
7 4 2 4 7 8 2 7 2 5
1 4 4 6 0 3 6 8 3 3
```

terminal 4: Run 5: TODO

图 7-30 生产者生产数据图

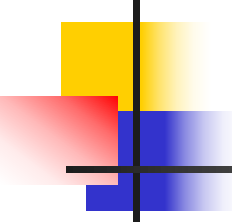
7.5.4 Kafka消息队列流

■ Kafka示例（消费者消费数据结果）

（6）执行KafkaWordCountConsumer代码，把收集到的随机数执行WordCount操作，结果如图7-31所示：

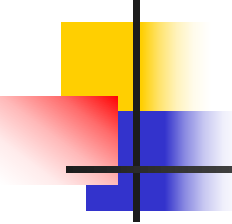
Time: 1542715610000 ms	Time: 1542715620000 ms	Time: 1542715630000 ms	Time: 1542715640000 ms
(4,88)	(4,125)	(4,162)	(4,203)
(8,78)	(8,109)	(8,148)	(8,177)
(6,67)	(6,91)	(6,135)	(6,178)
(0,74)	(0,117)	(0,159)	(0,194)
(2,104)	(2,134)	(2,177)	(2,224)
(7,86)	(7,126)	(7,159)	(7,206)
(5,87)	(5,139)	(5,178)	(5,221)
(9,88)	(9,121)	(9,159)	(9,201)
(3,84)	(3,122)	(3,164)	(3,195)
(1,84)	(1,116)	(1,159)	(1,201)

图 7-31 Kafka实例处理结果



7.6 Spark Streaming性能调优

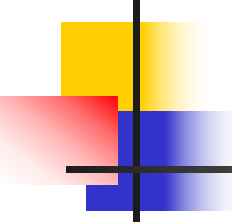
要使Spark Streaming应用程序在集群中获得最佳的性能实践，则需对一些性能参数进行调优。主要考虑两个方面：（1）有效使用集群资源，减少批处理所消耗的时间；（2）设置合理的窗口大小，使数据尽可能快速得到处理（即数据处理与数据接收节奏一致）。



7.6.1 减少批处理时间

优化运行时间可以降低每个批次数据的处理时间，
主要包括三个方面来实现：

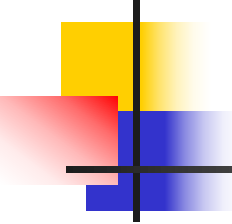
- 1、提高数据的接收并行度；
- 2、提高数据的处理并行度；
- 3、数据序列化。



7.6.1 减少批处理时间

- 提高数据接收并行度

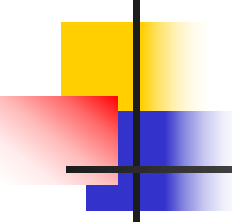
如果通过网络接收数据（比如Kafka、Flume、套接字等），则需要把数据反序列化并存储在Spark上，若数据接收成为瓶颈，则需要并行接收数据。在Worker节点上对每个输入的DStream创建一个接收器（Receiver）并运行，以接收数据流。通过创建多个DStream并配置从数据接收源接收不同分区的数据流。



7.6.1 减少批处理时间

- 提高数据接收并行度

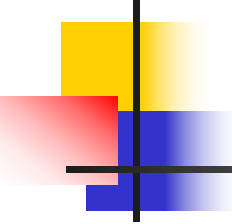
例如，接收两个数据主题的单个Kafka输入DStream可以分成两个Kafka输入流，每个输入流只接收一个主题。这将运行两个接收器，允许并行接收数据，从而提高整体吞吐量。多个DStream联合创建一个DStream，使用在一个输入DStream的转换操作作用在联合后的DStream上。



7.6.1 减少批处理时间

- 提高数据接收并行度

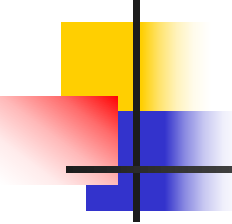
Receiver的RDD数据分区间隔由configuration parameter（配置参数）的 `spark.streaming.blockInterval` 决定。对于大多数的接收器，接收到的数据要合并成大的数据块，然后存储在Spark的内存中。每个批次的数量决定任务的数量，这些任务用来处理那些接收到的数据，即进行类“Map”操作，每个接收器每批次任务数目大约为（批时间间隔/块时间间隔）。



7.6.1 减少批处理时间

- 提高数据接收并行度

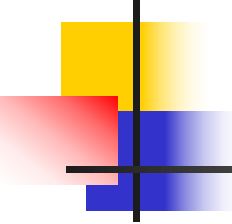
例如在**100**毫秒的块时间间隔将会在**2**秒的批次中创建**20**个任务。若任务太少，会导致有的核闲置，没有用来处理数据，使效率降低很多。对一个给定时间间隔的情况，如果要提升任务数，则需要降低每一小块的时间间隔。推荐的块时间间隔最小为**50**毫秒。



7.6.1 减少批处理时间

- 提高数据处理并行度

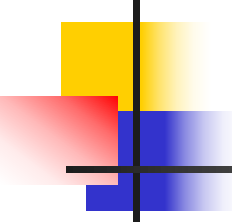
若在任务执行阶段并行的任务数量不多，会造成集群资源利用低下。例如分布式Reduce操作，如reduceByKey和reduceByKeyAndWindow，并行任务数量是在spark.default.parallelism中配置的。要确保均衡使用整个集群的资源，而不是把任务都集中在几个特定的节点上，对包含Shuffle的操作，增加并行度以确保更为充分使用集群资源。



7.6.1 减少批处理时间

■ 数据序列化

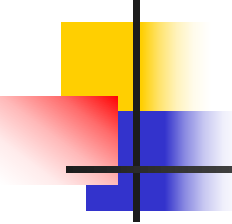
数据序列化的开销很大，特别是要实现亚秒级批次的大小，数据序列化主要包括下面两个方面。**RDD**数据的序列化，默认情况下**RDD**被保存为序列化子节数组，来减少**GC**停顿；输入数据序列化，将外部数据插入**Spark**，接收到的数据为子节型，需要反序列化为**Spark**的序列化格式。



7.6.1 减少批处理时间

- 数据序列化

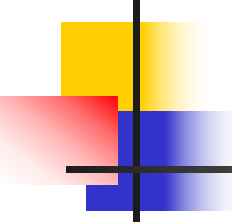
因此，输入数据的反序列化开销会成为一个瓶颈。**Spark Streaming**默认将接收到的数据序列化存储，以减少内存的使用。序列化和反序列化需要更多的**CPU**时间，更加高效的序列化方式（**Kryo**）和自定义的序列化接口可以更高效地使用**CPU**。



7.6.2 设置合适的批次大小

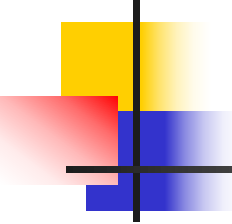
设置合适的批处理大小，首先要了解几个关键词。

- 1、批处理时间：每个批次的数据的处理时间。
- 2、批次间隔时间：两个批次处理的时间间隔。
- 3、数据速率：数据在集群上的处理速率。



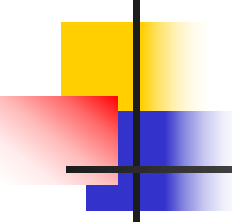
7.6.2 设置合适的批次大小

为了使群集上运行的**Spark Streaming**应用程序保持稳定，系统应该能够以接收数据的速度处理数据，即批处理数据应该在生成时尽快处理。处理数据速度对应批次处理时间，批次间隔时间对应数据流入速度。批次间隔时间应该大于批处理时间。



7.6.2 设置合适的批次大小

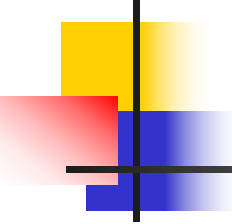
确定应用程序正确批次大小的好方法是使用保守的批处理间隔（例如，**5-10秒**）和低数据速率进行测试。要验证系统是否能够跟上数据速率，可以检查每个已处理批处理所遇到的端到端延迟的值（在**Spark**驱动程序**log4j**日志中查找**Total delay**（总延迟），或使用**StreamingListener**接口）。如果延迟保持小于或与批处理时间相当，则系统稳定。否则，如果延迟不断增加，代表系统处理速度跟不上数据输入速度。



7.6.3 优化内存使用

针对**Spark**应用程序的内存使用和**GC**（垃圾回收）行为，这一部分分为三个点，讲了自定义**Spark Streaming**应用的调优参数，来优化内存使用。

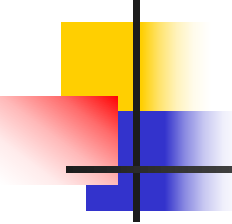
- 1、设置合理的**DStream**存储级别；
- 2、及时清理持久化的**RDD**；
- 3、并发垃圾收集。



7.6.3 优化内存使用

- 设置合理的DStream存储的级别

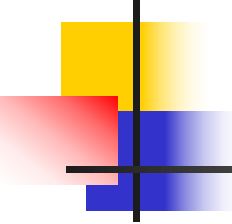
与RDD不同，RDD默认持久化级别是MEMORY_ONLY，而DStream默认持久化级别是MEMORY_ONLY_SER，尽管保持数据序列化会带来高序列化、反序列化开销，但是大大减少GC出现停顿的现象。



7.6.3 优化内存使用

- 及时清理持久化的RDD

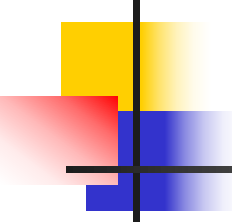
Streaming会将接收到的数据全部存储于可以用的内存中，因此应该及时清理已经处理完成的数据，以确保Streaming有足够的内存，默认情况下，所有Streaming持久化RDD的清理会使用内置的内存清理策略LRU（Least Recently Used）；通过设置`spark.cleaner.ttl`的值，Streaming就能自动地定期清除旧的内容。通过设置`spark.streaming.unpersist`属性启用内存清理，减少Spark RDD内存的使用，提升GC性能。



7.6.3 优化内存使用

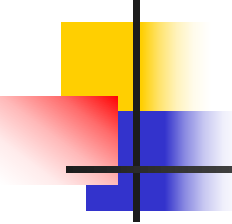
■ 并发垃圾收集

GC会影响任务的正常运行，任务执行时间的延长，会引起一系列不可预料的问题，采用不同的GC策略可以进一步减小GC对Job运行的影响。例如，使用并行mark-and-sweep GC能够减少GC的突然暂停情况，另外也可以以降低系统吞吐量为代价来获得最短GC停顿。



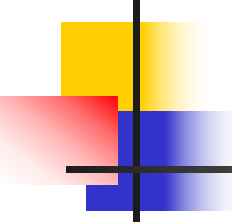
7.7 本章小结

本章主要介绍了**Spark Streaming**的运行原理，把连续的数据流切分成多个分段，并用**Spark Engine**处理这些分段，从而间接实现流处理功能；然后介绍了**Spark Streaming**的基础抽象**DStream**，其代表持续性的数据流；接着介绍了**DStream**的一系列操作方法，以及根据输入不同流类型介绍了四种**Spark Streaming**的示例；最后介绍**Spark Streaming**的性能优化方法。



思考与习题

1. 简要介绍静态数据与流数据及对它们的处理方法。
2. 简要介绍什么是DStream。
3. DStream中按批处理间隔划分的元素与RDD的关系？
4. Spark Streaming有哪三种基本输入源？
5. 什么是DStream的窗口操作？
6. 在Spark Streaming中为什么要设置合适的批次大小？



思考与习题

7. 简要介绍什么是有状态转换操作，什么是无状态转换操作？
8. 在IDEA中运行window操作。
9. 在IDEA中运行reduceByKeyAndWindow操作。
10. 在IDEA中运行updateStateByKey操作。
11. 结合7.5.4节配置并运行Kafka实例，并在Spark Streaming中处理Kafka流式数据。



Spark大数据 编程基础 (Scala版)

